

PRODUCTION OPERATIONS RUNBOOK

PostgreSQL on Kubernetes

Percona Operator — Private Cloud Deployment

Alert Troubleshooting & Incident Response Guide

Field	Value
Version	2.0
Platform	Kubernetes (Private Cloud) + Percona Operator for PostgreSQL
Monitoring	Prometheus + kube-prometheus-stack
Classification	INTERNAL — PRODUCTION USE ONLY
Maintained By	Database Platform Engineering Team
Last Updated	2026-03-04

1. Overview & Scope

This runbook provides step-by-step diagnostic and remediation procedures for the ten Prometheus alert categories observed in the production Kubernetes environment running PostgreSQL via the Percona Operator. It is designed for L1 SRE on-call engineers, L2 DBA leads, and L3 platform architects.

SCOPE

Kubernetes (Private Cloud) | Percona Operator for PostgreSQL | kube-prometheus-stack | Patroni HA | pgBackRest | etcd

1.1 Environment Assumptions

- Percona PostgreSQL Operator managing StatefulSets (PostgreSQL clusters)
- Patroni handling leader election over etcd
- pgBackRest for scheduled backups (S3-compatible or NFS target)
- Prometheus + Alertmanager deployed via kube-prometheus-stack Helm chart
- Private Cloud Kubernetes — no public internet egress by default
- HAProxy or MetalLB for load balancing; DR cluster in separate AZ/region
- Container runtime: containerd (or CRI-O) with cgroup v2 enabled
- Node OS: RHEL 8/9 or Ubuntu 22.04 LTS with kernel 5.15+

1.2 Alert Severity Reference

Severity	SLA / Response	Examples in This Runbook
CRITICAL	< 15 minutes	OOMKilled, etcd slow, StatefulSet mismatch, Backup failure
HIGH	< 30 minutes	NodeMemoryHigh, DiskIOSaturation, LB health probe
MEDIUM	< 2 hours	NodeMemoryMajorPageFaults, Node Exporter alerts

LOW	< 8 hours	Secrets expiry (advance warning)
-----	-----------	----------------------------------

IMPORTANT

Always start with the Quick Health Check (Section 2.3) before diving into alert-specific procedures. This establishes baseline cluster state and often reveals cascading failures.

2. Pre-Requisites & Access Setup

2.1 Required Tooling

- kubectl — configured with cluster admin or namespace-scoped DBA role
- helm — for operator and kube-prometheus-stack operations
- psql / pg_dump — PostgreSQL client tools (v14+ recommended)
- jq, curl, base64 — standard UNIX utilities
- Prometheus / Alertmanager UI or Grafana access
- pgBackRest CLI — on dedicated backup job pod or bastion
- etcdctl — v3.5+ for etcd cluster diagnostics
- stern (optional) — multi-pod log tailing for faster triage

2.2 Environment Variables (set before running commands)

```
# Set these variables at the start of every incident session
export NS="postgres"                # Percona operator namespace
export CLUSTER="pg-cluster"         # PerconaPGCluster name

export POD=$(kubectl get pods -n $NS -l \
  app.kubernetes.io/name=percona-postgresql \
  --field-selector=status.phase=Running \
  -o jsonpath="{.items[0].metadata.name}")

export LEADER=$(kubectl -n $NS exec -it $POD -- \
  patronictl -c /etc/patroni/patroni.yaml list \
  | grep Leader | awk '{print $2}')
```

2.3 Quick Health Check (run first on any alert)

Before investigating any specific alert, run this baseline health check. Cascading failures are common — an OOMKill can trigger StatefulSet mismatches, replication lag, and backup failures simultaneously.

```
# 1. Cluster pod status
kubectl get pods -n $NS -o wide

# 2. Patroni cluster state (verify leader + replicas)
kubectl -n $NS exec -it $POD -- patronictl -c /etc/patroni/patroni.yaml list

# 3. Recent events (look for warnings/errors)
kubectl get events -n $NS --sort-by=".lastTimestamp" | tail -30

# 4. Resource usage snapshot
kubectl top pods -n $NS
kubectl top nodes

# 5. Replication lag check (added)
kubectl -n $NS exec -it $POD -- psql -U postgres -c \
"SELECT client_addr, state, sent_lsn, replay_lsn FROM pg_stat_replication;"
```

3. Alert 1 — PostgreSQL Pods OOMKilled

CRITICAL

3.1 Alert Definition

Field	Value
Alert Name	PostgreSQL Pod OOMKilled
PromQL	kube_pod_container_status_last_terminated_reason{reason="OOMKilled", namespace=~"postgres"} > 0
Severity	CRITICAL
Typical Cause	Memory limit too low, runaway query, missing shared_buffers tuning
Impact	Pod restart causes connection drops; if leader killed, Patroni triggers failover

3.2 Immediate Triage

Step 1: Identify the OOMKilled pod and container:

```
kubectl get pods -n $NS -o json | jq '.items[] |  
  select(.status.containerStatuses[].lastState.terminated.reason=="OOMKilled")  
  | .metadata.name'
```

Step 2: Capture last termination details:

```
kubectl describe pod <pod-name> -n $NS | grep -A 10 "Last State"
```

Step 3: Check node-level memory pressure:

```
kubectl describe node <node-name> | grep -A 5 "Conditions"
```

Step 4: Review memory metrics via Prometheus:

```
# PromQL – container memory usage approaching limit
container_memory_working_set_bytes{namespace="postgres"}
  / container_spec_memory_limit_bytes > 0.9
```

Step 5 (Added): Check cgroup memory events for throttling evidence:

```
# Check cgroup v2 memory events on the node
kubectl debug node/<node-name> -it --image=ubuntu -- \
  cat /sys/fs/cgroup/kubepods/pod<pod-uid>/memory.events
```

3.3 Root Cause Identification

A. Check PostgreSQL Memory Parameters

```
kubectl -n $NS exec -it $POD -- psql -U postgres -c \
  "SHOW shared_buffers; SHOW work_mem; SHOW effective_cache_size;"
```

Rule of thumb: `shared_buffers` = 25% of pod memory limit. Be cautious with parallelism — `work_mem` × `max_parallel_workers` can OOM instantly.

B. Identify Runaway Queries

```
kubectl -n $NS exec -it $POD -- psql -U postgres -c "
SELECT pid, username, now() - query_start AS duration,
       query, state, wait_event_type, wait_event
FROM   pg_stat_activity
WHERE  state != 'idle' ORDER BY duration DESC LIMIT 20;"
```

C. Review PostgreSQL Logs

```
kubectl -n $NS logs <pod-name> --previous --tail=200 | \
  grep -iE "OOM|memory|fatal|killed"
```

D. Check Percona CR Memory Settings

```
kubectl -n $NS get perconapgcluster $CLUSTER -o yaml | grep -A 15 "resources"
```

E. Check Memory-Intensive Extensions (Added)

```
kubectl -n $NS exec -it $POD -- psql -U postgres -c \  
"SELECT name, setting FROM pg_settings WHERE name IN  
('shared_preload_libraries','max_locks_per_transaction');"
```

3.4 Remediation Steps

Step 1 — Emergency: Increase Memory Limit

```
kubectl -n $NS patch perconapgcluster $CLUSTER --type=merge -p '  
{  
  "spec": {  
    "instances": [{  
      "name": "instance1",  
      "resources": {  
        "limits": { "memory": "8Gi" },  
        "requests": { "memory": "6Gi" }  
      }  
    }  
  ] } }'
```

Step 2 — Tune PostgreSQL Parameters


```
# Edit the Percona ConfigMap or custom patroni config
kubectl -n $NS edit configmap <cluster>-patroni-config

# Recommended baseline for 8Gi pod:
shared_buffers = 2GB
work_mem = 64MB
maintenance_work_mem = 512MB
effective_cache_size = 6GB
max_connections = 200
temp_buffers = 32MB           # Added: limit per-session temp
```

Step 3 — Terminate Runaway Queries

```
kubectl -n $NS exec -it $POD -- psql -U postgres -c "
SELECT pg_terminate_backend(pid)
FROM   pg_stat_activity
WHERE  state != 'idle'
AND    query_start < now() - interval '10 minutes';"
```

Step 4 — Verify Pod Recovers

```
kubectl get pods -n $NS -w
kubectl -n $NS exec -it $POD -- patronictl -c /etc/patroni/patroni.yaml list
```

PREVENTION

Deploy `pg_oom_adj` or use Vertical Pod Autoscaler (VPA) in recommendation mode. Set resource limits conservatively and alert at 80% memory usage. Consider `pg_stat_statements` to identify memory-hungry queries. Configure `statement_timeout` and `idle_in_transaction_session_timeout`.

4. Alert 2 — NodeMemoryHighUtilization

HIGH

4.1 Alert Definition

Field	Value
Alert Name	NodeMemoryHighUtilization
PromQL	$(\text{node_memory_MemTotal_bytes} - \text{node_memory_MemAvailable_bytes}) / \text{node_memory_MemTotal_bytes} > 0.90$
Severity	HIGH
Threshold	> 90% for 5 minutes
Impact	Kernel OOM killer may terminate PostgreSQL or critical pods

4.2 Immediate Triage

Step 1: Identify top memory consumers:

```
kubectl top pods --all-namespaces --sort-by=memory | head -20
kubectl top nodes
```

Step 2: Check system-level memory:

```
kubectl debug node/<node-name> -it --image=ubuntu
free -h
cat /proc/meminfo | grep -E "MemTotal|MemFree|MemAvailable|Cached|SwapTotal"
```

Step 3: Check swap and PSI memory pressure:

```
swapon --show
cat /proc/pressure/memory    # PSI (kernel 4.20+), avg10 > 10 = contention
```

4.3 Root Cause Identification

A. Top Memory Processes & Slab Cache

```
ps aux --sort=-%mem | head -20
cat /proc/meminfo | grep -E "Slab|SReclaimable|SUnreclaim"
```

B. PostgreSQL shared_buffers vs OS Page Cache

High Slab/Cached memory is normal for PostgreSQL. True problem = low MemAvailable.

```
# PromQL to check real pressure
node_memory_MemAvailable_bytes{instance=~"<node>"}
/ node_memory_MemTotal_bytes{instance=~"<node>"}

```

C. Detect Memory Leaks in Operator or Sidecars

```
kubectl -n $NS top pods --containers
kubectl -n $NS get pods -o json | jq
  '.items[].status.containerStatuses[] | {name:.name, restartCount:.restartCount}'
```

4.4 Remediation Steps

Option A — Drop Reclaimable Kernel Caches

```
echo 1 > /proc/sys/vm/drop_caches    # pagecache only (safest)
echo 3 > /proc/sys/vm/drop_caches    # all (use with caution)
```

Option B — Evict Non-Critical Pods

```
kubectl cordon <node-name>
kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data --force
```

Option C — Check Connection Counts / PgBouncer

```
kubectl -n $NS exec -it $POD -- psql -U postgres -c \
"SELECT count(*) as active_connections FROM pg_stat_activity;"
kubectl get pods -n $NS | grep pgbouncer
```

IMPORTANT

Never enable swap on Kubernetes PostgreSQL nodes. Swapping causes severe performance degradation. Configure `vm.swappiness=1` on all DB nodes. Consider memory QoS with cgroup v2 for guaranteed memory.

5. Alert 3 — NodeMemoryMajorPageFaults

MEDIUM

5.1 Alert Definition

Field	Value
Alert Name	NodeMemoryMajorPageFaults
PromQL	rate(node_vmstat_pgmajfault[5m]) > 500
Severity	MEDIUM (escalates to HIGH if sustained > 30 min)
Meaning	Pages read from disk into RAM — memory pressure indicator

A major page fault occurs when the kernel must read a page from disk. For PostgreSQL, this means data evicted from shared_buffers or OS page cache, causing latency spikes.

5.2 Triage Steps

```
vmstat 2 10          # look for 'majflt' column
sar -B 2 10          # pgscand/s = pages scanned by kswapd

perf stat -e major-faults -p $(pgrep postgres | head -1) sleep 10

kubectl -n $NS exec -it $POD -- psql -U postgres -c "
SELECT checkpoints_timed, checkpoints_req, buffers_checkpoint,
       buffers_clean, maxwritten_clean, buffers_backend
FROM   pg_stat_bgwriter;"
```

5.3 Remediation

A. Tune PostgreSQL I/O Parameters

```
bgwriter_lru_maxpages = 100
bgwriter_delay = 200ms
checkpoint_completion_target = 0.9
shared_buffers = 25-40% of total pod memory
effective_io_concurrency = 200 # for SSD/NVMe
```

B. Huge Pages & Swap

```
cat /proc/meminfo | grep Huge
# PostgreSQL: huge_pages = try

swapoff -a
sed -i '/ swap /d' /etc/fstab
```

6. Alert 4 — NodeDiskIOSaturation (dm-0)

HIGH

6.1 Alert Definition

Field	Value
Alert Name	NodeDiskIOSaturation
PromQL	rate(node_disk_io_time_weighted_seconds_total{device="dm-0"}[5m]) > 1
Device	dm-0 = device mapper (LVM volume backing PG data dir)
Severity	HIGH

dm-0 is a Linux device-mapper device, typically an LVM logical volume. When I/O utilization exceeds 100%, every PostgreSQL disk operation experiences latency.

6.2 Immediate Triage

```
iostat -x dm-0 2 10 # %util, await, svctm

kubectl -n $NS exec -it $POD -- psql -U postgres -c "
SELECT schemaname, relname, heap_blks_read, heap_blks_hit,
       idx_blks_read, idx_blks_hit
FROM   pg_statio_user_tables
ORDER BY heap_blks_read DESC LIMIT 20;"
```

6.3 Root Cause & Remediation

A. Check Autovacuum / Bulk Operations

```
kubectl -n $NS exec -it $POD -- psql -U postgres -c "  
SELECT pid, query_start, state, query FROM pg_stat_activity  
WHERE query ILIKE '%vacuum%' OR query ILIKE '%COPY%';"
```

B. Throttle Autovacuum I/O

```
kubectl -n $NS exec -it $POD -- psql -U postgres -c "  
ALTER SYSTEM SET autovacuum_vacuum_cost_delay = '20ms';  
ALTER SYSTEM SET autovacuum_vacuum_cost_limit = 200;  
SELECT pg_reload_conf();"
```

C. Tune WAL/Checkpoint & Expand Volume

```
max_wal_size = 4GB    checkpoint_completion_target = 0.9  
wal_compression = on    synchronous_commit = local  
  
lvextend -l +100%FREE /dev/mapper/dm-0  
resize2fs /dev/mapper/dm-0
```

BEST PRACTICE

Separate WAL (pg_wal) onto a dedicated volume. WAL is sequential write — exclusive I/O bandwidth significantly reduces contention. Consider `io_uring` (PostgreSQL 16+) for improved async I/O.

7. Alert 5 — etcdMemberCommunicationSlow

CRITICAL

CRITICAL

etcd is the backbone of Patroni HA. Slow or unreachable members cause spurious failovers, split-brain, and service disruption.

7.1 Alert Definition

Field	Value
PromQL	histogram_quantile(0.99, rate(etcd_network_peer_round_trip_time_seconds_bucket[5m])) > 0.15
Secondary	up{job="etcd"} == 0 (member unreachable)
Impact	Patroni leader election disrupted — potential PG downtime

7.2 Immediate Triage

```
kubectl -n kube-system exec -it etcd-<node> -- \
  etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  member list -w table

etcdctl endpoint health --cluster -w table
etcdctl endpoint status --cluster -w table
```

7.3 Root Cause & Remediation

A. Network / Disk Latency

```
ping -c 20 <control-plane-node-2-ip>
iperf3 -c <target-ip> -t 30

# etcd WAL fsync latency (> 10ms p99 = problem)
histogram_quantile(0.99, rate(etcd_disk_wal_fsync_duration_seconds_bucket[5m]))
```

B. Defragment & Compact

```
for EP in $(etcdctl member list | awk '{print $5}'); do
    etcdctl defrag --endpoints=$EP
    sleep 5
done

REV=$(etcdctl endpoint status --write-out=json | jq '.[0].Status.header.revision')
etcdctl compact $REV
```

C. Restore Unhealthy Member

```
etcdctl member remove <member-id>
rm -rf /var/lib/etcd/member
etcdctl member add <n> --peer-urls=https://<ip>:2380
```

D. Verify Patroni Stability

```
kubectl -n $NS exec -it $POD -- patronictl -c /etc/patroni/patroni.yaml list
# Confirm: one Leader, all replicas streaming
```

8. Alert 6 — KubeStatefulSetReplicasMismatch

CRITICAL

8.1 Triage

```
kubectl get statefulsets -n $NS
kubectl describe statefulset <ss-name> -n $NS | grep -A 20 "Events"
kubectl get pods -n $NS -o wide
kubectl -n $NS exec -it $POD -- patronictl -c /etc/patroni/patroni.yaml list
kubectl logs -n $NS deployment/percona-postgresql-operator --tail=100 | \
  grep -iE "error|warn|replica"
```

8.2 Root Cause

```
# Pod stuck pending — resource constraints
kubectl get pod <pending-pod> -n $NS -o yaml | grep -A 20 "conditions"

# PVC not bound
kubectl get pvc -n $NS
kubectl describe pvc <pvc-name> -n $NS

# Image pull / init container failures
kubectl describe pod <pod> -n $NS | grep -iE "pull|image|registry"
kubectl -n $NS logs <replica-pod> | grep -iE "bootstrap|patroni|error"
```

8.3 Remediation

```
# Fix A – Delete stuck pod (StatefulSet recreates it)
kubectl delete pod <stuck-pod> -n $NS

# Fix B – Force Patroni re-initialize replica
kubectl -n $NS exec -it $POD -- \
  patronictl -c /etc/patroni/patroni.yaml reinit $CLUSTER <replica>
```

9. Alert 7 — Node Exporter Alerts

MEDIUM

9.1 Common Alerts

Alert	Threshold	Typical Cause
NodeCPUHighUsage	> 90% for 15m	Runaway query, parallel query overload
NodeFileSystemAlmostOutOfSpace	< 10% free	WAL bloat, temp files, core dumps
NodeFileSystemFilesFillingUp	< 5% inodes	Many small temp files, WAL segments
NodeNetworkReceiveErrors	> 0.01/s	NIC driver issues, MTU mismatch
TargetDown	up == 0	node-exporter crashed / DaemonSet issue

9.2 Triage & Fixes

```
# DaemonSet & pod health
kubectl get daemonset -n monitoring | grep node-exporter
kubectl get pods -n monitoring -l app.kubernetes.io/name=node-exporter -o wide
kubectl logs -n monitoring <node-exporter-pod> --tail=50

# Check taints blocking DaemonSet
kubectl describe node <node-name> | grep Taint
```

FileSystem Out of Space

```
kubectl debug node/<node-name> -it --image=ubuntu
du -sh /host/var/lib/postgresql/data/* 2>/dev/null | sort -rh | head -20

# Clean old WAL – verify replication slots first
kubectl -n $NS exec -it $POD -- psql -U postgres -c "
SELECT slot_name, active, restart_lsn FROM pg_replication_slots;"
# Drop inactive slots: SELECT pg_drop_replication_slot('<slot>');
```

10. Alert 8 — Load Balancer Health Probe Issues (DR)

HIGH

10.1 Triage

```
kubectl get svc,endpoints -n $NS

kubectl run test-conn --image=postgres:14 --rm -it -- \
  pg_isready -h <service-ip> -p 5432 -U postgres

kubectl -n $NS exec -it $POD -- \
  curl -s http://localhost:8008/health | python3 -m json.tool

kubectl -n $NS exec -it $POD -- patronictl -c /etc/patroni/patroni.yaml list
kubectl get networkpolicy -n $NS
```

10.2 Remediation

Reconfigure LB + DR Verification

```
# HAProxy backend (Patroni health on port 8008)
backend postgres_primary
  option httpchk GET /primary HTTP/1.0
  http-check expect status 200
  server pg1 <pod-ip>:5432 check port 8008

# DR replication check
kubectl -n $NS exec -it $POD -- psql -U postgres -c "
SELECT client_addr, state, sent_lsn, replay_lsn,
       (sent_lsn - replay_lsn) AS replication_lag
FROM   pg_stat_replication;"
```

11. Alert 9 — Backup Job Failures

CRITICAL

RPO RISK

A failed backup means your Recovery Point Objective is at risk. If the next backup also fails, you may not meet recovery SLAs. Escalate to L2 DBA immediately if manual retry also fails.

11.1 Triage

```
kubectl get jobs -n $NS | grep -i backup
kubectl describe job <backup-job> -n $NS
kubectl -n $NS exec -it $POD -- pgbackrest --stanza=db info
kubectl -n $NS exec -it $POD -- cat /var/log/pgbackrest/backup.log | tail -100
```

11.2 Root Cause & Fixes

```
# A. Repository connectivity
kubectl -n $NS exec -it $POD -- pgbackrest info --log-level-console=detail
kubectl get secret -n $NS | grep pgbackrest

# B. Stanza init
kubectl -n $NS exec -it $POD -- pgbackrest --stanza=db stanza-create
kubectl -n $NS exec -it $POD -- pgbackrest --stanza=db check

# C. WAL archive check
kubectl -n $NS exec -it $POD -- psql -U postgres -c "
SELECT archived_count, failed_count, last_failed_wal
FROM   pg_stat_archiver;"

# D. Remove stale lock + manual backup
kubectl -n $NS exec -it $POD -- rm -f /tmp/pgbackrest/db-backup.lock
kubectl -n $NS exec -it $POD -- pgbackrest --stanza=db --type=full backup
```


BEST PRACTICE

Retention: full=2, diff=7. Schedule full weekly, differential daily, WAL continuously. Test restores quarterly. Use pgBackRest verify for ongoing integrity.

12. Alert 10 — API Keys / Secrets Expiry

LOW

Escalates to HIGH (< 7 days), CRITICAL (expired).

12.1 Audit Secrets

```
kubectl get secrets -n $NS \
  -o custom-columns=NAME:.metadata.name,TYPE:.type,CREATED:.metadata.creationTimestamp

# TLS cert expiry check
for SECRET in $(kubectl get secrets -n $NS \
  -o jsonpath='{.items[?(@.type=="kubernetes.io/tls")].metadata.name}'); do
  echo "=== $SECRET ==="
  kubectl get secret $SECRET -n $NS -o jsonpath='{.data.tls\.crt}' \
    | base64 -d | openssl x509 -noout -enddate
done
```

12.2 Rotation Procedures

```
# PostgreSQL password rotation
NEW_PASS=$(openssl rand -base64 32)
kubectl create secret generic <pg-secret> -n $NS \
  --from-literal=password=$NEW_PASS --dry-run=client -o yaml | kubectl apply -f -
kubectl -n $NS exec -it $POD -- psql -U postgres -c \
  "ALTER USER postgres PASSWORD '$NEW_PASS';"
kubectl -n $NS annotate perconapgcluster $CLUSTER percona.com/issue-restart="$(date +%s)"

# TLS cert renewal (cert-manager)
kubectl delete secret <tls-secret> -n $NS # cert-manager re-creates
```

12.3 Proactive Monitoring

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
spec:
  groups:
    - name: certificate-expiry
      rules:
        - alert: TLSCertExpiringSoon
          expr: x509_cert_expiry - time() < 30 * 86400
          labels: {severity: warning}
        - alert: TLSCertCritical
          expr: x509_cert_expiry - time() < 7 * 86400
          labels: {severity: critical}
```

13. Escalation Matrix

Alert	L1 (SRE On-Call)	L2 (DBA Lead)	L3 (Platform Arch)
OOMKilled	Restart pod, check mem	Tune PG params, raise limits	Capacity planning, VPA
NodeMemoryHigh	Cordon/drain node	Identify leak, tune buffers	Node pool expansion
DiskIOSaturation	Cancel vacuum, check WAL	I/O tuning, pg_wal sep.	Storage architecture
etcd Slow/Down	Check pods, defrag	Member recovery	etcd cluster rebuild
StatefulSet Mismatch	Delete stuck pod	Patroni reinit	Operator upgrade
Backup Failure	Check logs, retry	Fix repo, restore test	RPO breach notify
Secrets Expiry	Rotate, restart pods	Audit all creds	Vault integration
LB / DR Issues	Check endpoints	Replication check	DR arch review

ESCALATION GUIDELINES

L1 escalates to L2 after 15 min without resolution for CRITICAL, 30 min for HIGH. L2 escalates to L3 if root cause unidentified within 1 hour. Update incident channel every 15 min.

14. Post-Incident Checklist

14.1 Immediate Recovery Verification (within 1 hour)

- All PostgreSQL pods Running and Ready
- Patroni: exactly one Leader, all configured replicas Streaming
- Backup job executed and completed successfully after incident
- etcd cluster healthy (all members reachable, no pending elections)
- Prometheus targets all UP (node-exporter, postgres-exporter)
- Replication lag < 1 second on all standbys
- No active OOMKilled pods or CrashLoopBackOff
- DR site replication current and verified (Added)
- Client applications reconnected and healthy (Added)

14.2 Verification Commands

```
# Full health snapshot
kubectl get pods,statefulsets,pvc,svc -n $NS
kubectl -n $NS exec -it $POD -- patronictl list
kubectl -n $NS exec -it $POD -- psql -U postgres -c
    "SELECT * FROM pg_stat_replication;"
kubectl -n $NS exec -it $POD -- pgbackrest --stanza=db info
kubectl top nodes && kubectl top pods -n $NS
```

14.3 Post-Incident Review Actions

- Document incident timeline in your incident tracker
- Capture Prometheus metrics screenshots (memory, I/O, replication lag)
- Update this runbook with any new findings or improved commands
- File a Jira/ticket for permanent fixes (resource limits, tuning)
- Schedule postmortem within 48 hours for P1 incidents
- Review Alertmanager thresholds — alert fire too late or early?

- Validate DR replication is current and DR cluster can promote
- Review and update capacity forecasts based on incident data (Added)
- Verify monitoring dashboards captured the incident timeline (Added)

IMPORTANT

Every CRITICAL incident should have a 5-Why root cause analysis documented within 5 business days. This prevents recurrence and improves runbook quality. Assign an owner for each action item with a due date.

— END OF RUNBOOK —

PostgreSQL on Kubernetes — Percona Operator — Production Runbook v2.0